

Database Architect Take-Home Assignment: "The Global Consolidation Initiative"

Scenario Overview

Our organization currently operates a fragmented database estate:

- **Legacy Tier:** 50+ MySQL instances (on-prem) running critical but aging applications.
- **Core Tier:** A high-transaction SQL Server & Postgres cluster handling the most matured platform.
- **Future Tier:** A growing PostgreSQL footprint (AWS RDS) intended to become the primary standard for our new go-to platform.

The Task

You are the DB Architect. You must submit a technical proposal (max 4-5 pages or a slide deck) addressing the following three theoretical pillars:

Part 1: Architectural Strategy & Migration

We are migrating a high-traffic brand from the **On-prem MySQL** environment to **PostgreSQL on AWS RDS**.

1. **Schema Evolution:** How do you handle the conversion of MySQL-specific features (e.g., **SET** types, different locking behaviors) to PostgreSQL?
2. **Zero-Downtime Migration:** Propose a migration strategy that minimizes cutover downtime. Compare using **AWS DMS** vs. a **native logical replication** approach.
3. **Hybrid Connectivity:** How would you architect the networking between the on-prem MySQL servers (legacy platform) and the new Cloud RDS (new platform) to ensure low latency and high security?

Part 2: Performance & Scalability (The "IOPS Ceiling")

During a recent data purging exercise on a PostgreSQL RDS instance, the team hit an **IOPS limit of 5,000**, causing application degradation.

1. **Tuning:** What specific PostgreSQL parameters or architectural changes would you implement to allow background purging without impacting front-end performance?
2. **Scaling:** Explain when you would choose **RDS Proxy** vs. application-side **Connection Pooling** (like PgBouncer) to manage 10,000+ concurrent connections.

Part 3: Automation, Security & Governance

1. **Infrastructure as Code (IaC):** Describe your approach to standardizing database deployments across different flavors (MySQL, PG, SQL Server) using Terraform or Ansible. How do you ensure "Golden Images" for on-prem servers?
2. **Data Purging Automation:** Design a workflow (using Cron, AWS Lambda, or Jenkins) that identifies and purges data older than 2 years across all platforms while maintaining an audit trail.
3. **Security Awareness:** A developer requests "Temporary Admin Access" to a Production SQL Server to "fix a bug." Propose a **Just-In-Time (JIT)** access workflow that satisfies security compliance while not blocking the developer.

Then the hands-on pillar:

Part 4: Database Architect Hands-On Challenge

1. Assignment Overview

GiG is currently transitioning its legacy "flat" data structures to modern, partitioned architectures to improve performance and manageability across **MySQL, PostgreSQL, and SQL Server**.

Your task is to demonstrate how you would automate the deployment, population, and "live refactor" of a high-traffic database table.

2. The Lab Environment

You are free to use a local VM, a Docker container, or a cloud instance (AWS/Azure).

- **Target Engine:** Choose **one** of the following: PostgreSQL 16+, MySQL 8.0+, or SQL Server 2022.
- **Automation Tools:** You must use **Ansible** or **PowerShell** for the infrastructure and logic.

3. The Requirements

Task A: Infrastructure as Code (IaC)

Automate the installation of the database engine and the creation of a database named **GIG_REFACTOR_LAB**.

- The setup must be **idempotent** (running the script twice should not cause errors).
- Configure the instance for basic security (e.g., non-default port or a specific admin user).

Task B: Data Generation

Automate the creation of a "Legacy" table: `transaction_logs_flat`.

- **Schema:** `id` (PK), `created_at` (Timestamp), `brand_id` (Int), `payload` (JSON or Text).
- **Volume:** Populate this table with **1,000,000 rows** of randomized data spanning the last **12 months**.

Task C: The Migration & Archiving Simulation

Your goal is to transition from the "Flat" table to a "Partitioned" architecture while simulating a production environment with limited IOPS.

1. **New Schema:** Create a new table `transaction_logs_modern` partitioned by **Month** (based on `created_at`).
2. **Batch Migration:** Automate the migration of the **last 3 months** of data from the flat table to the partitioned table.
 - **Throttling:** You must move data in **batches** (e.g., 25,000 rows).
 - **Safety:** Implement a "sleep" or "delay" between batches to ensure you do not saturate disk I/O.
3. **Finalize:** Once the 3-month window is safely migrated, **drop** the legacy `transaction_logs_flat` table to simulate a successful archive/purge.

4. Evaluation Criteria

Criteria	What we are looking for
Automation Quality	Use of variables, error handling, and clean code structure.
Partitioning Logic	Correct use of native partitioning features.
Performance Awareness	Evidence that you considered I/O impact.
Validation	A final check in your script that verifies row counts between the source and destination.

5. Submission Instructions

- Provide a **GitHub Repository link** or a **ZIP file** containing your code.
- Include a **README.md** with:
 - Instructions on how to run your automation.
 - A brief explanation of your batching strategy.

P.S. Kindly make sure that everything works in order to demonstrate your hands-on challenge to us in your next interview.